

EDACurry: A Multi-Language Framework to Create, Translate, and Manipulate Transistor-Level Netlists

Nicola Dall’Ora, *Member, IEEE*, Enrico Fraccaroli, *Member, IEEE*,
Renaud Gillon, *Senior Member, IEEE*, Franco Fummi, *Member, IEEE*

Abstract—The Simulation Program with Integrated Circuit Emphasis (SPICE), introduced in 1973, laid the foundation for transistor-level netlist representation. Over the decades, numerous SPICE dialects emerged, each tailored to specific needs but sharing core semantics. This article presents EDACurry, an innovative, open-source framework designed to simplify and automate the creation, translation, and manipulation of transistor-level netlists across SPICE-based languages such as Eldo and Spectre. By leveraging a unified internal Abstract Syntax Tree (AST), EDACurry bridges syntactical differences while maintaining semantic consistency, enabling efficient manipulation workflows. The framework supports custom operations, including design space exploration, defect model injection, and subcircuit wrapping, all accessible through Python. Demonstrated applications highlight its versatility in analog design workflows, offering integration with commercial and open-source simulators. With EDACurry, analog designers gain a powerful tool to support netlist manipulation tasks, improving productivity and fostering innovation in circuit design. The entire framework is publicly available on GitHub: <https://github.com/esd-univr/EDACurry>.

Index Terms—Analog circuits, transistor-level models, SPICE-based simulation, netlist manipulation, design space exploration.

I. INTRODUCTION

The parent of transistor-level design languages is the Simulation Program with Integrated Circuit Emphasis (SPICE) [1], which established the foundation for describing electronic circuits at the transistor level. Following SPICE, many proprietary design languages emerged to specify transistor-level netlists [2]. Different Electronic Design Automation (EDA) vendors extended the SPICE language to incorporate new features and components tailored to their specific design workflows and needs [3]. These extensions often include active components such as different classes of Metal-Oxide-Semiconductor Field-Effect Transistors (MOSFETs), which enable more accurate modeling of complex physical behaviors.

As each vendor developed its proprietary language and the tools that use them, the EDA market became increasingly fragmented. As a result, these languages are often incompatible outside their respective environments. That said, despite

syntactic differences, most transistor-level languages share a standard semantic structure. This similarity allows designers to map each netlist component to its corresponding schematic circuit accurately. This process ultimately aids in physically implementing an IC for various applications, including automotive, aerospace, and healthcare.

This work introduces a framework that allows designers to manipulate transistor-level netlists without relying on proprietary software. Building upon a prototype introduced by the authors in 2021 [4], the framework has been significantly enhanced to support various design complexities and address practical design needs. The resulting tool, named EDACurry, enables manipulation of netlists described in various formats and generates output compatible with both commercial (e.g., Siemens Eldo, Cadence Spectre, Synopsys HSPICE, Keysight ADS) and open-source (e.g., LTSpice, Xyce, NGSpice, Qucs) analog simulators.

EDACurry is implemented in C++ and integrates with Python via the Pybind11 library [5], which facilitates bidirectional interoperability between C++ and Python. This structure supports rapid prototyping and allows users to script and manipulate netlists using the rich Python ecosystem, ranging from graphical visualization tools to advanced Multimodal Large Language Model (MLLM)-based models.

The innovative contributions of the EDACurry manipulation framework are summarized below:

- Identification of a standard semantic structure across multiple transistor-level languages, used to construct a unified internal Abstract Syntax Tree (AST) representation.
- Implementation of front-ends for SPICE, Eldo, Spectre, XML, and JSON formats that map input netlists to the internal AST.
- Development of back-ends that export the AST as SPICE, Eldo, Spectre netlists, or XML and JSON files.
- Support for XML- and JSON-based save/load functionality after each manipulation step, enabling the creation of custom manipulation workflows tailored to specific design tasks.
- The framework is demonstrated across four key applications: design space exploration, injection of diverse classes of defect models, export of analog circuits in XML and JSON formats, and graphical tree-based visualization of netlists for manual inspection.

The remainder of this paper is organized as follows: Section II introduces the transistor-level languages targeted by the

N. Dall’Ora and E. Fraccaroli and F. Fummi are with the Department of Engineering for Innovation Medicine (Section of Engineering and Physics), University of Verona, Verona, Italy. E-mail: name.surname@univr.it. N. Dall’Ora is also with the Department of Engineering Sciences, Guglielmo Marconi University, Rome, IT. E-mail: n.dallora@unimarconi.it.

R. Gillon is with the Sydelity B.V., Kruisem, Belgium. E-mail: renaud.gillon@sydelity.com

Manuscript received TBD; revised TBD.

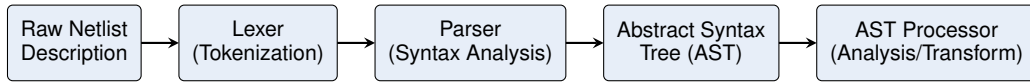


Figure 1. Parsing pipeline, from raw netlist input through lexical and syntactic analysis to a structured AST, ready for tooling and transformation.

framework and presents the technical foundations of netlist parsing and representation. Section III presents the state of the art of manipulation tools for transistor-level languages. Section IV details the architecture and components of the EDACurry framework. Section V presents use cases illustrating the framework's flexibility and utility in practice. Finally, Section VI summarizes the contributions and outlines directions for future work.

II. BACKGROUND AND TECHNICAL FOUNDATIONS

This section introduces the key characteristics of transistor-level Hardware Description Languages (HDLs) commonly used to model analog circuits. It then presents the technical foundations required for automated netlist manipulation, focusing on the parsing process, internal representations such as ASTs, and parser generators like ANOther Tool for Language Recognition (ANTLR).

A. Transistor-level Languages

Circuit simulation allows manufacturers to validate analog designs before fabrication, reducing time-to-market and production costs. The design of complex analog electronic circuits can be rapidly evaluated and validated on circuit simulators. Transistor-level simulators such as SPICE, Eldo, and Spectre model analog circuits' internal structure and behavior [6]. Since the initial release of SPICE in 1973 [1], numerous extensions have emerged, including Spice 2, Spice 3, PSpice [7], and HSpice. Eldo, developed by Mentor Graphics, and Spectre, developed by Cadence, are proprietary extensions of the SPICE language. Both languages support fundamental SPICE analyses like DC, AC, and transient and co-simulation with Verilog-A and Verilog-AMS modules [8]. Due to their advanced features and simulation capabilities, Spectre and Eldo are widely adopted in industry.

Both languages support a variety of analyses, including frequency and transient simulations, and often deliver better performance than other SPICE-based simulators. Although their syntaxes differ, Spectre and Eldo share a standard semantic model that governs analog circuit simulation behavior. All these languages allow designers to write netlists in plain text using a defined syntax, which can then be simulated using an analog simulator. However, automating the manipulation of these languages remains challenging, prompting the need for a unified approach to parsing and editing netlists.

Tasks such as parsing proprietary syntax, converting between dialects, or programmatically modifying circuit structures often require ad hoc and non-reusable code. Addressing these challenges requires a systematic approach to netlist parsing and transformation. To this end, the EDACurry framework was developed as a language-independent tool for parsing,

editing, and transforming transistor-level netlists. This approach's core is the use of parsers and structured intermediate representations.

B. Parsing and internal representation of netlists

The standard approach to automatically reading and manipulating languages, such as those describing transistor-level circuits, relies on parsers. Parsers are programs that can examine a sequence of input strings using a particular set of rules or grammar. They convert input strings into structured representations, typically trees or graphs, that reflect the hierarchical organization of the source language. Once converted into an intermediate format, the input can be programmatically analyzed and manipulated. Common intermediate representations include ASTs, parse trees, and hierarchical trees. EDACurry uses a AST as its internal representation, offering a simple and efficient way to model netlist structure. An AST captures the hierarchical syntactic structure of a program in tree form.

Parsing is a fundamental step in automatically reading and transforming netlist descriptions. Rather than operating directly on raw text, a parser converts the input into a structured representation that reflects the syntactic and semantic organization of the language. This is typically done in two stages: lexical analysis and syntactic analysis. The lexer scans the input text and breaks it into tokens, distinct language elements such as keywords, identifiers, or numerical values. These tokens are then passed to the parser, which assembles them into a tree-like structure according to the language's grammar rules. The resulting structure, known as an AST, captures the hierarchical relationships among circuit elements and enables systematic manipulation of netlists. Figure 1 illustrates this process, from raw text to structured AST representation, highlighting the key stages of parsing and transformation.

A grammar, which defines the structure of a language, is conceptually divided into two main components: the *lexer* and the *parser*. The lexer identifies groups of characters in the input stream and classifies them as tokens. In the context of HDL languages, tokens typically represent circuit elements, parameters, expressions, or structural symbols. The parser then takes these tokens and matches them to patterns defined in the grammar. The parser builds a corresponding structure in the AST when a valid sequence is identified. If an invalid token sequence is encountered, the parser raises an error and may attempt recovery using heuristics.

To automate this process, EDACurry uses the ANTLR library to generate front-ends for parsing netlists based on formal grammar definitions [9]. ANTLR is a widely used open-source parser generator that processes structured text and produces language recognition and transformation components. Given a grammar, ANTLR generates the lexer, parser, and auxiliary classes such as visitors and listeners. In EDACurry,

the visitor class traverses the parsed token stream, as guided by the parser, and constructs the internal AST representation for subsequent manipulation. The open-source nature of ANTLR ensures that the framework remains free from proprietary dependencies and easily extensible.

III. RELATED WORK: MANIPULATION TOOLS FOR TRANSISTOR-LEVEL LANGUAGES

This section reviews tools developed to manipulate transistor-level netlists, from proprietary commercial utilities to academic prototypes and open-source frameworks. Each tool targets specific use cases, such as netlist translation, parameter tuning, or simulation automation, and supports different subsets of SPICE-derived languages. By analyzing their functionalities, limitations, and design assumptions, we position EDACurry within the broader landscape of netlist manipulation tools.

A. Commercial netlist manipulation tools

The Spectre to Eldo (spect2el) converter is a utility in the Siemens EDA tool suite [10]. This tool translates Spectre syntax into Eldo-compatible netlists, generating mapping files to convert component and parameter names. It also caches intermediate files to optimize performance and avoids reconversion unless the input netlist changes. While spect2el simplifies the integration of Spectre designs into Eldo workflows, it supports only unidirectional translation, from Spectre to Eldo, and does not allow conversion in the opposite direction. Furthermore, the tool does not support structural manipulation or language abstraction, as its functionality is limited strictly to syntax translation for simulation purposes.

The Netlist Translator for SPICE and Spectre is a Keysight tool for converting SPICE or Spectre netlists into formats compatible with the Advanced Design System (ADS) [11]. It is intended to support the design and simulation of physical-layer components. It can generate ADS schematics directly from Spectre or SPICE netlists. It also converts model files for integration into ADS design kits. While primarily designed for translation and simulation integration, the tool also supports command-line workflows that enable advanced users to inspect and edit intermediate files before schematic generation. However, these manipulations are manual and not framework-based, and the tool does not provide reusable mechanisms for structural changes or language-agnostic transformations.

IBM has developed an advanced suite of transistor-level tools to support custom analog and mixed-signal circuit design [12]. These tools import SPICE netlists and convert them into a unified internal representation known as the Integrated Data Model (IDM), which supports flat and hierarchical circuit views. Operating within IBM's Nutshell execution environment, they enable structural netlist manipulation, parameter tuning, and hierarchical abstraction through dynamically loadable modules and Tcl scripting. This modular and scriptable design allows designers to perform fine-grained optimization and analysis while maintaining compatibility with manufacturing workflows. However, these tools are proprietary and unavailable for external use, limiting their accessibility and integration outside IBM's design ecosystem.

B. Academic Tools and Research Prototypes

Several academic efforts have explored tools for manipulating SPICE netlists, often focusing on parameter tuning and simulation automation.

Batas et al. [13] introduced a Python interface that integrates with SPICE-based simulators to support parameter-driven circuit design. The interface enables designers to import parameterized netlists, adjust input variables dynamically, and automate simulations at both the device and circuit levels. Simulation results, including waveform data, operating point values, and derived metrics, are exposed in Python for interactive analysis and optimization. The system supports advanced tasks such as multi-dimensional parameter sweeps and inversion of electrical constraints into geometric parameters (e.g., determining W from I_D and V_{DS}). This interface was the foundation for Spicedim [14], a graphical tool that extends this functionality with a GUI and waveform viewer to support CMOS analog circuit sizing. While both tools offer robust parameter control and simulation automation, they do not support structural manipulation of netlists, such as modifying topology or subcircuit hierarchy. Additionally, they are not open-source and are bound to the syntax and assumptions of standard SPICE dialects.

Li et al. [15] introduce AutoTEA, an automated transistor-level optimization tool designed for GRM FPGA architectures. AutoTEA focuses on optimizing subcircuits within interconnect (INT) and logic (CLB) blocks by extracting key netlists, modeling area and load characteristics, and applying iterative transistor sizing to minimize a global cost function, such as the area-delay product. The tool generates simulation-ready SPICE netlists with accurate performance models, with improvements over manually optimized designs. While AutoTEA automates sizing and cost-driven refinement of transistor-level circuits, its transformation logic is embedded within a fixed optimization flow and does not support user-defined structural manipulation. Furthermore, the tool is not open-source, limiting its adaptability and integration into broader design environments.

The SPICE Manipulation Language (SML) [16] is a domain-specific language designed to simplify the creation of SPICE netlists through a higher-level, more intuitive syntax. SML introduces object-oriented constructs and list-based abstractions, enabling users to define circuit elements and their connections programmatically. It supports standard components such as resistors, capacitors, inductors, and transistors, and allows control structures to generate repetitive or parameterized circuit configurations. SML serves as a wrapper that translates its code into SPICE-compatible netlists, aiming to reduce the complexity associated with manual SPICE coding. However, SML lacks advanced features for structural netlist manipulation, such as dynamic topology alterations or hierarchical design abstractions. The tool has not been publicly released, limiting its accessibility and evaluation within the broader community.

C. Open-Source Tools for Netlist Manipulation

This subsection reviews open-source tools for manipulating transistor-level netlists, focusing on their capabilities and limitations.

NETGEN [17] is an open-source tool for SPICE netlist manipulation, primarily used for performing Layout vs. Schematic (LVS) checks. It compares the electrical connectivity and components between two circuit representations, typically a layout-derived netlist and a schematic-derived one, to verify structural consistency and design integrity. In addition to SPICE, NETGEN supports multiple netlist formats, including SIM, NTK, and ACTEL, enhancing its applicability across toolchains. Based on Tcl, its scripting interface allows users to define device matching rules, configure port mappings, and programmatically control comparison strategies. While this scripting capability enables powerful and customizable LVS workflows, NETGEN is not designed for general-purpose structural manipulation or transformation of netlists. It lacks abstractions such as ASTs or dialect-independent representations, and its workflows remain tightly focused on comparison rather than circuit generation or synthesis. The entire library is distributed under an open-source license¹.

PySpice [18] is a Python library that interfaces with NGSpice and Xyce to enable programmatic circuit definition and simulation of existing SPICE netlists. Users can define circuit topologies using an object-oriented API, directly specifying components, models, and simulation parameters in Python. The library also includes a partial SPICE parser for importing netlists and supports integration with KiCad schematics through the KiCadTools extension. Simulation results, such as voltages and currents, are returned as NumPy arrays and can be visualized using Matplotlib or exported to CSV and JSON formats for further analysis. PySpice also supports interactive use in Jupyter notebooks and offers tools for batch simulation and parameter sweeps. Despite these features, PySpice does not support structural netlist manipulation, such as modifying circuit topology or abstracting subcircuits, and it lacks a unified internal representation for multi-dialect support. The library is fully open-source and available under a permissive license².

SPICEPilot [19] is an open-source framework that automates SPICE code generation and circuit simulation using Large Language Models (LLMs). Built atop the PySpice infrastructure, it enables users to describe circuit specifications in natural language and automatically generate Python-based SPICE code for analog and digital circuits. The framework includes a structured benchmarking system that classifies circuits by complexity, from simple single-transistor designs to large-scale multi-stage circuits. It evaluates them based on simulation compliance, functional accuracy, and analysis completeness. It also features error-detection and correction tools to refine LLM-generated circuits and incorporates data augmentation from open-source sources to enhance training diversity. Although SPICEPilot simplifies circuit design entry and supports simulation, it does not provide a structural

netlist manipulation framework or offer tools for modifying or abstracting existing circuit hierarchies. The tool is fully open-source and intended as a research platform for LLM-guided circuit generation and verification³.

Dan Fritchman's Netlist library [20] is a Python package designed for reading, manipulating, and writing popular circuit netlist formats. It emphasizes device models and supports hierarchical and modular netlists, allowing users to define components like modules, signals, and instances declaratively or programmatically. The tool accepts input formats such as SPICE, HSPICE, and Spectre, and outputs detailed representations of circuit hierarchies and interconnections in a Xyce-compatible format. This versatility makes Netlist adaptable to various design and simulation workflows, bridging high-level abstractions and detailed, implementation-ready outputs within Python. Despite its flexibility in input-output handling, the tool does not support structural netlist transformations or dialect-independent abstractions beyond format-specific parsing and generation. The library is released under an open-source license⁴.

Hdl21 [21], also developed by Dan Fritchman, is a Python-based hardware description library tailored for analog and custom integrated circuit design. It provides a flexible environment for creating and managing circuit designs, enabling users to define components, interconnections, and parameters programmatically using Python syntax. The library emphasizes modularity and reuse, with constructs like Module, Signal, and Instance facilitating hierarchical design. Hdl21 supports parameterized components, making it suitable for scalable designs and simulation scenarios. It integrates with Process Design Kits (PDKs), allowing incorporation of fabrication-specific details, and supports SPICE-class netlist generation compatible with popular analog simulators. Hdl21 enhances reproducibility, version control, and automation by replacing traditional schematic capture workflows with code-based design. However, it does not offer parsing or transformation capabilities for existing netlists, and its focus remains on code-based design entry rather than manipulation of legacy circuit descriptions. The library is fully open-source and accessible to the community⁵.

Nimphel [22] is a Python framework developed by Gutiérrez et al. for modular and parametric generation of SPICE netlists. The framework offers comprehensive input and output functionalities that support electronic design workflows. Currently, Nimphel supports parsing Spectre-format SPICE netlists into Python objects, enabling users to modify, extend, or reuse existing designs efficiently. It allows dynamic import of parameters from files or PDKs using custom parsers, facilitating flexible adaptation of designs to different technologies and constraints. On the output side, Nimphel can export designs to various SPICE formats, ensuring compatibility with simulators like NGSpice and Spectre. Additionally, it supports generating custom text formats, enabling integration with specialized tools and workflows. However, the framework currently sup-

¹<https://github.com/RTimothyEdwards/netgen>

²<https://github.com/PySpice-org/PySpice>

³<https://github.com/ACADLab/SPICEPilot>

⁴<https://github.com/dan-fritchman/Netlist>

⁵<https://github.com/dan-fritchman/Hdl21>

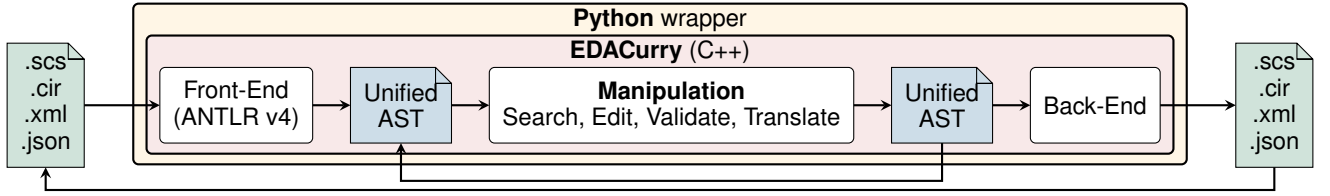


Figure 2. Structure of the proposed framework for manipulating transistor-level analog circuits.

ports only Spectre-format input, and while it enables parameter editing, it lacks features for dialect abstraction across SPICE variants. The library is available as open-source software⁶.

D. Comparison and positioning of EDACurry

While numerous tools have been developed for transistor-level netlist manipulation, most are limited in scope and tailored to specific simulation dialects or narrowly defined tasks. Some tools focus on parameter tuning through simulation scripts, others offer high-level circuit definition languages, and a few support format-specific parsing and code generation. However, none of these approaches provide a unified, language-independent framework capable of structurally manipulating netlists across SPICE dialects. EDACurry addresses this gap by introducing an AST representation that captures shared semantics across formats like SPICE, Eldo, and Spectre. This internal representation enables cross-dialect transformation, structural editing, and reusable automated manipulations across design contexts. EDACurry also supports serializing manipulation routines, allowing designers to build custom, repeatable workflows for modifying and inspecting circuit descriptions.

EDACurry is not intended to replace analog simulators and does not perform circuit simulation itself. Instead, it serves as a pre-simulation manipulation platform designed to operate between design capture and simulation, facilitating the transformation and refinement of circuit netlists. Its modular and scalable architecture enables analog designers to automate everyday editing tasks, inject fault models, wrap subcircuits, flatten hierarchies, and explore design variants programmatically. By exposing its internal logic to Python via C++ bindings, EDACurry integrates seamlessly with the broader data science and design automation ecosystems. This positions EDACurry not just as a tool, but as a flexible framework for creating customized netlist manipulation pipelines aligned with modern analog open-source design workflows.

IV. THE EDACURRY FRAMEWORK

This section presents *EDACurry*, a language-independent framework for parsing, analyzing, and transforming transistor-level netlists across multiple dialects, with a programmable interface in Python. The framework is released as open-source software under the MIT license and is publicly available on GitHub⁷. A high-level overview of the framework architecture, including its front-ends, internal representation, and output

back-ends, is shown in Figure 2 and will be detailed in the following sections. The framework supports multiple front-ends for parsing heterogeneous input formats, which include SPICE, Eldo, Spectre, XML, and JSON, into a unified internal AST representation. This unified representation can then be traversed and manipulated. For example, by editing parameters, validating structure, translating between dialects, or searching for specific subcomponents. Once manipulations are complete, back-ends generate output netlists in the desired language, like SPICE, Eldo, Spectre, JSON, or XML.

JSON support, introduced as a key extension over our previous prototype [4], enables serialization of the internal representation. This makes it possible to chain manipulation routines into reusable processing pipelines, facilitating automation and collaborative workflows across teams and design stages.

The core of EDACurry is implemented in C++ for performance, and exposed to Python for ease of use and rapid development. Using the provided bindings, designers can build manipulation pipelines either in C++ or interactively via Python scripts. The following sections describe the framework’s internal architecture, language integration, manipulation model, and applications relevant to analog circuit designers. Together, these design choices make EDACurry suitable for a wide range of use cases in analog design workflows.

A. Parsing front-ends and grammar design

As discussed in Section II-B and shown on the left side of Figure 2, EDACurry uses ANTLR v4 to generate its parsing front-ends. Currently, two grammars have been developed using ANTLR syntax: one covering Eldo and SPICE, and another for Spectre. Because Eldo’s syntax extends that of SPICE, both languages can be parsed using the same grammar.

Front-ends serve as the entry point for ingesting netlist descriptions into the framework. Front-ends for transistor-level languages are built using the ANTLR library, while others (e.g., for JSON or XML) rely on separate C++ libraries, detailed in the EDACurry repository. ANTLR generates a set of C++ parser classes that serve as the basis for front-end development. These skeleton parsers can be visitors or listeners, implemented as top-down recursive-descent visits. ANTLR supports both visitor and listener modes: visitors call a `visit()` method per element, while listeners use `enter()` and `exit()` hooks. EDACurry adopts the visitor approach to transform the set of organized tokens into the internal AST representation. These front-ends have been validated against a diverse set of open-access transistor-level analog circuits, detailed in the project documentation.

⁶<https://github.com/servinagrero/nimphel>

⁷<https://github.com/esd-univr/EDACurry>

1) ANTLR grammar design for SPICE-derived languages:

To support the parsing of transistor-level netlists, EDACurry defines dedicated ANTLR grammars for Eldo and Spectre. Although both dialects descend from SPICE and share a similar overall purpose, their syntaxes differ significantly in structure, conventions, and supported keywords. Eldo features a relatively well-defined and deterministic syntax, making its grammar concise and unambiguous. In contrast, Spectre includes optional constructs and inline commands that introduce grammatical ambiguity and necessitate more complex disambiguation rules [23], [24]. As a result, the Spectre grammar in EDACurry is more extensive and modular, isolating language-specific constructs for clarity and maintainability.

Each grammar follows ANTLR's two-stage architecture, with a lexer that tokenizes the input and a parser that organizes tokens into a structured syntax tree. This approach ensures a robust parsing pipeline that feeds into a shared visitor, which constructs a unified internal representation of the circuit.

The grammars were designed to maintain a consistent high-level structure, such as defining netlists as sequences of subcircuits, models, analyses, and control statements, thereby enabling alignment in the subsequent mapping stage. This dialect-specific grammar modularity enables EDACurry to robustly support parsing from heterogeneous netlist formats, a critical step before uniform manipulation can occur.

The following section details how this parsed structure is translated into a language-independent AST used internally by EDACurry.

2) *Semantic mapping across dialects:* While Eldo and Spectre differ significantly in syntax, they share a foundational set of semantic concepts that govern how analog circuits are described and simulated. These include modular subcircuits, component instantiation, analysis types (e.g., .AC, .TRAN), and control directives (e.g., SAVE, ALTER). EDACurry leverages these commonalities to map parsed netlists into a shared, language-independent AST for uniform manipulation.

The process of mapping is performed using dialect-specific ANTLR visitors. These visitors walk the parse trees generated by each grammar and populate the unified AST by interpreting each syntactic construct according to its semantic role. This design allows EDACurry to abstract away language-specific quirks while preserving the circuit's structural and functional meaning. Some of these mappings are summarized in Table I, highlighting how equivalent constructs across Eldo and Spectre are unified under common AST node types.

Table I
SEMANTIC ALIGNMENT BETWEEN ELDO AND SPECTRE CONSTRUCTS,
MAPPED TO UNIFIED AST ELEMENTS.

Eldo Syntax	Spectre Syntax	Mapped AST Node
.SUBCKT myAmp in out vdd R1 in out 1k	subckt myAmp (in out vdd) R1 (in out) resistor r=1k	Subckt Component
.AC DEC 10 1k 1Meg	ac dec 10 1k 1Meg	Analysis
.SAVE all	save all	Control

For instance, both dialects use subcircuit instantiations that begin with a component identifier (typically prefixed with X), followed by a list of connection nodes and a reference to the subcircuit type. The specific ordering and syntax differ,

but semantically these constructs serve the same role and are mapped to the same Component node in the AST.

```
X1 in out myAmp GAIN=1000
```

Listing 1. Subcircuit instantiation in Eldo.

```
X1 (in out) myAmp gain=1000
```

Listing 2. Equivalent subcircuit instantiation in Spectre.

The Eldo and Spectre examples in Listings 1 and 2, respectively, ultimately populate a Component node with the same logical content: instance name X1, nodes in and out, and master myAmp. This translation process is visualized in Figure 3, which shows an excerpt of the unified AST structure used to represent these semantically equivalent constructs.

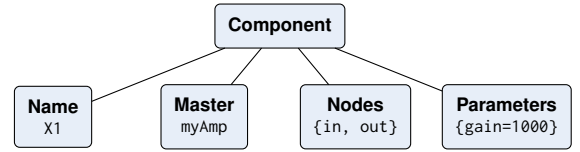


Figure 3. Excerpt of the unified AST representing the subcircuit instantiation in both Eldo and Spectre.

However, not all constructs are semantically portable. One notable example is Eldo's .MEAS directive, which performs post-simulation measurements and computations. Spectre lacks a direct equivalent for this construct, making lossless translation impossible. Attempts to convert a .MEAS directive to Spectre results in a controlled translation error.

In some cases, equivalent behavior can be approximated using a combination of Spectre constructs. For example, save and print statements, expression-based calculations via calc, or post-processing using the Open Command Environment for ANalysis (OCEAN) scripting language can replicate .MEAS-like functionality. However, these translations are context-dependent and not guaranteed to preserve full intent.

This semantic mapping layer ultimately enables EDACurry to act as a multi-dialect manipulation engine. Aligning diverse syntaxes to a unified internal representation allows transformations, analyses, and validations to be implemented once and reused across dialects.

B. Framework internal representation

The internal representation of the EDACurry framework serves as the central abstraction layer between front-ends and back-ends, as illustrated in Figure 2. This structure is more detailed in Figure 4, which presents a simplified UML diagram of the language-independent AST used internally. Each class in this diagram corresponds to a core construct in a transistor-level netlist, such as subcircuits, components, and analysis directives. The grammars for Eldo and Spectre are designed to translate source netlists into this shared structure, abstracting over syntax differences. This unified model draws inspiration from HIFSuite [25], which similarly unified multiple hardware description languages into a common intermediate form.

A typical netlist is represented as a Circuit object, which contains multiple Subckt instances representing subcircuits.

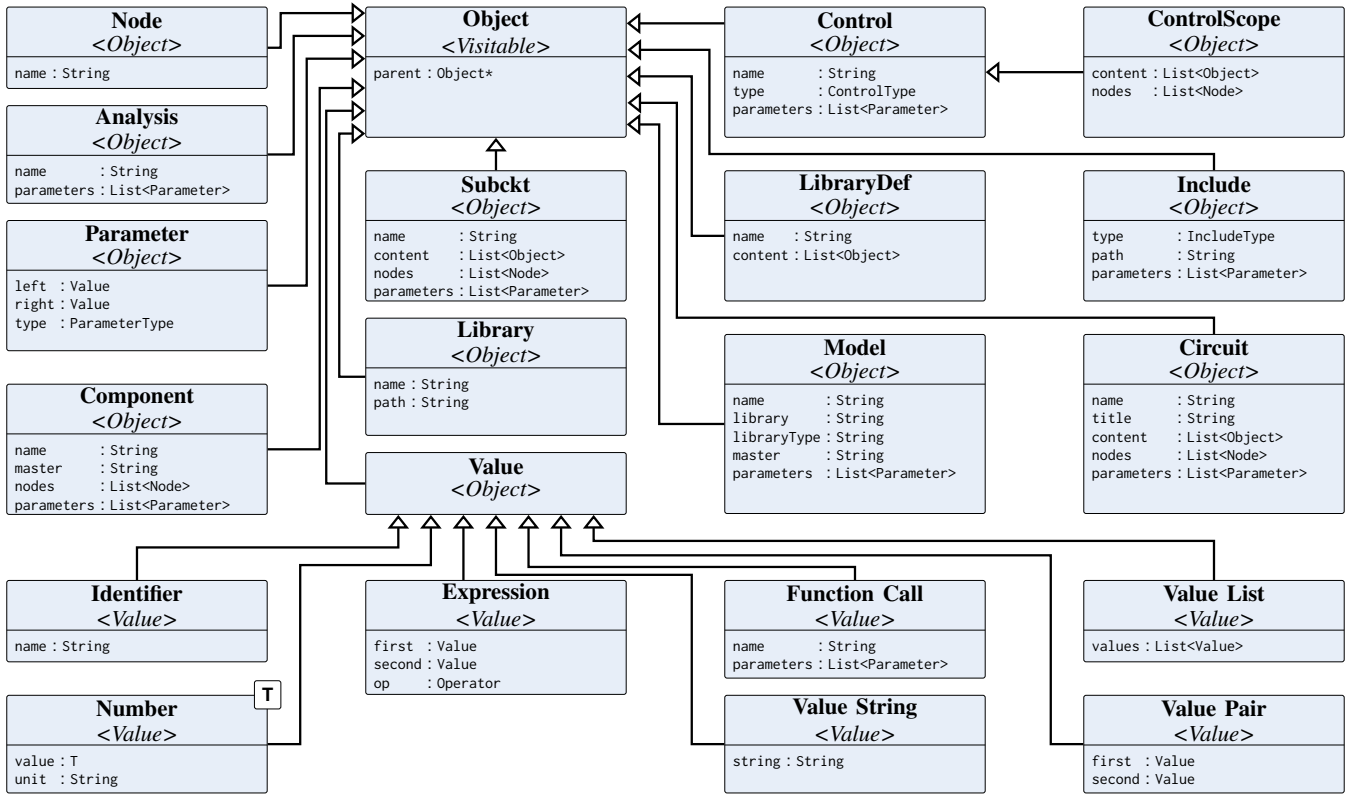


Figure 4. UML Class diagram of the internal structure of the EDACurry framework. The Object class is the parent class, and all the other classes are a specialization of the parent class.

Each subcircuit includes a set of Component objects, each of which may have associated Parameter values. Pins are represented by the Node class, which defines connectivity between components. The internal structure also includes classes for models, control commands, analysis blocks, and other constructs common to analog descriptions, as shown in Figure 4. The UML diagram abstracts the AST structure by focusing on class composition and inheritance, omitting runtime-specific values, thereby offering a conceptual blueprint of how analog netlists are modeled internally.

All classes inherit from a base Object class, enabling polymorphism and structural consistency across the AST. The framework uses the *Composite* design pattern to enable hierarchical grouping: for example, a Circuit contains a list of nodes, parameters, and nested objects. It also adopts the *Visitor* pattern by implementing an `accept()` function in each class, facilitating recursive traversal and transformation. These patterns ensure that manipulation logic can be modularly applied to any element in the netlist hierarchy.

To expose this C++ framework to Python, all classes are wrapped using Pybind11 [5], a header-only library for creating seamless C++/Python bindings. This allows users to script manipulations interactively in Python while leveraging the performance of a compiled core.

The following section demonstrates how this internal structure enables powerful manipulation routines applied to analog netlists.

C. AST-based netlist manipulation

The core part of the EDACurry framework is the manipulation functions available through the framework API. As depicted in Figure 2, the manipulation functions are in the framework’s middle. In particular, the manipulation functions are designed to operate on the internal representation to manipulate the original analog circuit read through the front ends. These manipulation functions can change, add, or remove every statement composing a transistor-level analog circuit.

All these manipulations on an analog circuit are possible due to the internal structure of the framework represented as an AST. On the internal AST, users can easily visit each element of the tree to search, add, or remove internal parts of the AST. These manipulations can be applied automatically to the internal representation of the netlist so that the user can easily modify the circuit without having to edit the netlist manually as with regular text editors. For example, through the manipulation functions, it is possible to:

- check the correctness of internal components of a sub-circuit, e.g., inspect if each component has an associated master;
- rename all the elements that compose a netlist, e.g., components, nodes, parameter values;
- inspect a PDK to locate where a particular subcircuit or model is defined;
- perform parameter sweeping by generating netlist variants where key parameters are systematically varied;
- substitute specific component models (e.g., replacing one

MOSFET variant with another across the netlist);

- extract a subcircuit subtree for reuse, modular analysis, or export as a standalone block.

```
.SUBCKT OPAMP1 inn inp out pd xpd vdda vssa
xd1 vssa vdda prim_nwd AREA=6.13907e-9 PJ=476.6e-6 M=1
xi0 vssa net69 prim_nd AREA=1e-12 PJ=4e-6 M=1
xi3 net51 vdda prim_pd AREA=790e-15 PJ=3.3e-6 M=1
xr0 vssa vdda net51 prim_rdiffp L=2.1e-6 W=700e-9 M=1
xr1 vdda vdda net69 prim_rdiffp L=2.1e-6 W=700e-9 M=1
xcc01 net12 out prim_cpoly AREA=4.7e-9 ...
mnm12 net13 net56 vssa vssa nmos1 ...
mnm11 net56 net56 vssa vssa nmos1 ...
mnb02 net27 net27 net21 vssa nmos1 ...
mnpd1 net21 xpd vssa vssa nmos1 ...
mn001 out net13 vssa vssa nmos1 ...
mnpd2 net13 pd vssa vssa nmos1 ...
mnc01 net13 net69 net12 vssa nmos1 ...
mpb02 net27 net51 net158 vdda pmos1 ...
mppd1 net158 xpd vdda vdda pmos1 ...
mps11 net31 net158 vdda vdda pmos1 ...
mpd11 net56 inn net31 net31 pmos1 ...
mp001 out net158 vdda vdda pmos1 ...
mpd12 net13 inp net31 net31 pmos1 ...
mpb01 net158 net158 vdda vdda pmos1 ...
.ENDS
```

Listing 3. OPAMP subcircuit described with the Eldo language.

```
subckt OPAMP1 inn inp out pd xpd vdda vssa
xd1 (vssa vdda) prim_nwd AREA=6.13907e-9 PJ=476.6e-6 m=1
xi0 (vssa net69) prim_nd AREA=1e-12 PJ=4e-6 m=1
xi3 (net51 vdda) prim_pd AREA=790e-15 PJ=3.3e-6 m=1
xr0 (vssa vdda net51) prim_rdiffp l=2.1e-6 w=700e-9 m=1
xr1 (vdda vdda net69) prim_rdiffp l=2.1e-6 w=700e-9 m=1
xcc01 (net12 out) prim_cpoly ...
mnm12 (net13 net56 vssa vssa) nmos1 ...
mnm11 (net56 net56 vssa vssa) nmos1 ...
mnb02 (net27 net27 net21 vssa) nmos1 ...
mnpd1 (net21 xpd vssa vssa) nmos1 ...
mn001 (out net13 vssa vssa) nmos1 ...
mnpd2 (net13 pd vssa vssa) nmos1 ...
mnc01 (net13 net69 net12 vssa) nmos1 ...
mpb02 (net27 net51 net158 vdda) pmos1 ...
mppd1 (net158 xpd vdda vdda) pmos1 ...
mps11 (net31 net158 vdda vdda) pmos1 ...
mpd11 (net56 inn net31 net31) pmos1 ...
mp001 (out net158 vdda vdda) pmos1 ...
mpd12 (net13 inp net31 net31) pmos1 ...
mpb01 (net158 net158 vdda vdda) pmos1 ...
ends OPAMP1
```

Listing 4. OPAMP subcircuit described with the Spectre language.

To illustrate the manipulation capabilities of EDACurry, we consider an OPAMP circuit extracted from the IEEE analog benchmarks suite [26]. This OPAMP model is a typical analog building block that amplifies low-level signals. We present two implementations of this circuit: one in Eldo (see Listing 3) and another in Spectre (see Listing 4). Both descriptions are written in different dialects and represent the same analog netlist. While the Eldo and Spectre implementations differ in syntax, their underlying structure is semantically equivalent. For instance, Spectre uses parentheses to group component pins, whereas Eldo uses a flat space-separated format. Nonetheless, both versions describe circuits with identical behavior, demonstrating the framework's ability to normalize syntax while preserving semantics.

Starting from the descriptions in Listing 3 and Listing 4, the two OPAMP netlists are parsed using the dedicated front-ends provided in the EDACurry framework. Although these front-ends operate on distinct dialects, Eldo and Spectre, they both produce an equivalent internal representation. Figure 5 illustrates the unified AST structure representing the xd1

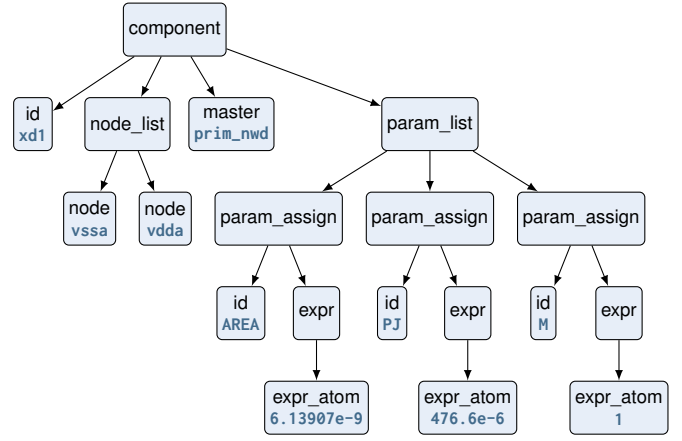


Figure 5. Excerpt of the unified AST representing the OPAMP1 subcircuit.

component shared between the two descriptions. This AST encodes the same logical content, instance name, pin connections, master reference, and parameter values, regardless of syntactic differences in the source language. Thanks to this unified representation, EDACurry enables a wide range of manipulations to be applied uniformly across dialects. For instance, we could rename the xd1 instance, change the order of its pins, update parameter values, or even remove the component entirely, all from the same internal tree.

We now present three examples using the Python API to make these capabilities concrete. These demonstrate how designers can programmatically inspect, modify, and augment netlists using the unified AST. For simplicity, error checking is omitted, but should be included in practice.

```
# Step 1: Search the component by name.
component = edacurry.find_component(root, "xd1")
# Step 2: Search the parameter by name.
parameter = edacurry.find_parameter(component, "M")
# Step 3: Change the parameter value.
parameter.right = edacurry.Double(2.45)
```

Listing 5. Updating the M parameter of component xd1 via AST manipulation.

In Listing 5, a specific parameter of the xd1 component is located and its value is updated directly in the AST. Such targeted edits are helpful in automated tuning, design-space exploration, or post-layout correction flows. The search functions in the framework, such as `find_component` and `find_parameter`, require specifying the starting point in the AST hierarchy. Typically, the root of the parsed netlist is used, but any internal node can be provided to restrict the search to a subtree if desired.

```
# Step 1: Find and rename OPAMP1 to OPAMP1_WP.
original = edacurry.find_subckt(root, "OPAMP1")
original.name = "OPAMP1_WP"
# Step 2: Create wrapper subckt OPAMP1 with same ports.
wrapper = edacurry.Subckt("OPAMP1")
for node in original.nodes:
    wrapper.nodes.append(edacurry.Node(node.name))
# Step 3: Create a component that instantiates OPAMP1_WP.
xopamp = edacurry.Component("XOP1", "OPAMP1_WP")
for node in original.nodes:
    xopamp.nodes.append(edacurry.Node(node.name))
wrapper.content.append(xopamp)
# Step 4: Add the wrapper back to the root
root.content.append(wrapper)
```

Listing 6. Wrapping an existing subcircuit (OPAMP1) into a new subcircuit for hierarchical reuse or probing.

The second example, Listing 6, demonstrates a structural transformation: the existing OPAMP1 subcircuit is renamed and encapsulated inside a new subcircuit with the original name. This approach enables non-invasive instrumentation or hierarchical reuse without modifying the core subcircuit, a common requirement in analog design workflows. Wrapping strategies are instrumental in extending PDK components, enabling internal node probing, and integrating auxiliary structures for fault injection and S-parameter analysis. Depending on the designer's goal, wrappers can be constructed externally, preserving original internal nodes but adding external ports, or internally, renaming internal nodes (e.g., using conventions such as `dut_<node>`) to simplify connectivity and integration of probes or monitors. Such flexible manipulation of circuit topology is complex to achieve manually but becomes seamless with the EDACurry API.

```
# Step 1: Find the component X1.
original = edacurry.find_component(root, "X1")
# Step 2: Change the master to a new one.
original.master = "OPAMP2_NEW"
```

Listing 7. Changing the master of a component to redirect its instantiation.

In Listing 7, the master attribute of the component X1 is updated to point to a new subcircuit. This operation is typical in workflows involving hierarchical abstraction, where modified versions of a subcircuit (e.g., for fault injection or testing) must be referenced without altering the original instantiating context.

D. Back-ends and pipelining

The EDACurry framework includes back-ends that translate the internal representation into various output formats, as shown on the right side of Figure 2. These formats include transistor-level languages like Eldo, Spectre, and SPICE and intermediate representations such as JSON and XML.

This multi-format support enables *pipelining*, where manipulation routines are chained across multiple stages. Each stage performs a targeted transformation or analysis and passes its output to the next via a serialized intermediate format. JSON and XML are particularly well suited for this role due to their simplicity, speed, and compatibility with modern scripting environments and legacy infrastructure. JSON is often used in Python-based workflows for integration with visualization tools, testing frameworks, and design-space exploration engines. In contrast, XML is typically adopted in GUI-based environments or mixed-language toolchains requiring schema validation and structured parsing.

The back-end infrastructure is designed to be modular and extensible. Users can easily write new exporters targeting custom formats or third-party simulation environments, further expanding EDACurry's applicability. Most AST constructs are faithfully preserved during conversion to JSON or XML, ensuring structural consistency across pipeline stages. However, dialect-specific constructs, such as conditional simulation directives, may require additional extensions or custom handling.

To illustrate a typical pipeline, consider a workflow where an input OPAMP circuit is parsed from Spectre and converted into XML. This XML file is passed to a fault injection tool

built on EDACurry, which inserts .ALTER statements or perturbs model parameters. The modified netlist is returned and re-exported to a SPICE dialect for simulation. Such workflows can consist of one or multiple tools, all interoperating through standardized intermediate formats.

V. EDACURRY FRAMEWORK APPLICATIONS

The proposed manipulation framework allows designers to create and manipulate transistor-level netlists with automated procedures that exploit the EDACurry API, available through C++ or Python. The framework is flexible and extensible, enabling the creation of pipelines for complex manipulation tasks that would otherwise be tedious and error-prone if performed manually. For example, the framework can read SPICE, Eldo, Spectre, JSON, or XML descriptions as input, apply manipulations, and generate new descriptions containing both the original netlist information and the applied modifications, as illustrated in Figure 2. All elements of the original transistor-level analog circuit are mapped into the internal AST via the dedicated front-ends (see Section IV-A), allowing efficient, structured manipulation.

In the following, we present several tasks that an analog designer could perform with EDACurry:

- 1) design space exploration;
- 2) injection of different classes of defect models;
- 3) printing a netlist as an XML or JSON file;
- 4) graphically representing netlists as trees.

These tasks will be detailed in the following subsections, explaining the implementation and the practical utility for analog designers. These examples illustrate the framework's potential in real-world scenarios and aim to stimulate further ideas for its application. These examples are not exhaustive; EDACurry's flexibility allows designers to develop additional custom manipulation routines as needed.

A. Design space exploration

One of the main applications of the EDACurry framework is *design space exploration*. This process enables designers to systematically investigate a circuit's design alternatives to meet specific performance requirements. For analog designers, creating a new circuit is a complex task that demands a deep understanding of circuit behavior and precise performance prediction. Design space exploration is crucial for achieving optimal trade-offs between power consumption, area, speed, noise, and reliability. Traditionally, this process is time-consuming, involving extensive simulations and manual parameter tuning [27].

The EDACurry framework facilitates design space exploration by allowing direct manipulation of the circuit's internal representation. Designers can modify component values, parameters, or even interconnections to assess their impact on performance. For example, using the framework's API, it is possible to automatically adjust component parameters like resistances or capacitances, or systematically alter the circuit topology by adding or removing components. This enables designers to build pipelines that automate exploration tasks, including parameter sweeps and sensitivity analyses.

```

import numpy as np, edacurry, os
# If the output directory does not exist, create it.
if not os.path.exists("eldo_result"):
    os.mkdir("eldo_result")
# Parse the ELDO file.
root = edacurry.parse_eldo("eldo/circ_base/opamp.cir")
# Sweep width parameters from 1um to 5um, 5 points.
width_values = np.linspace(1.0, 5.0, 5)
# Find the subcircuit "OPAMP1".
subckt = edacurry.find_subckt(root, "OPAMP1")
# Find all NMOS components inside the subcircuit.
comps = edacurry.find_components_by_master(subckt, "nmos1")
# For each width value...
for width in width_values:
    # Update each NMOS width.
    for nmos in comps:
        # Find the parameter "W".
        w_param = edacurry.find_parameter(nmos, "W")
        # Update the parameter value.
        w_param.right = edacurry.Double(width)
    # Output each variant netlist.
    filename = f"eldo_result/variant_w_{width:.1f}.cir"
    with open(filename, "w") as f:
        f.write(edacurry.write_eldo(root))

```

Listing 8. Updating the W parameter of all nmos1 components.

For instance, in Listing 8, the designer updates the value of the width parameter W of all nmos1 components. Automated adjustment of such key parameters is helpful in analog sizing, gain tuning, or trade-off analysis during early design phases. For simplicity, we will omit the output generation in the following listings.

Beyond basic parameter tuning, EDACurry also supports more complex exploration scenarios, such as device substitution. For instance, different transistors, such as NMOS, PMOS, NPN, or PNP, can be systematically swapped to assess the impact on performance.

```

import numpy as np, edacurry, os
# Parse the ELDO file.
root = edacurry.parse_eldo("eldo/circ_base/opamp.cir")
# Traverse the AST and find all NMOS components based on
# master name pattern.
nmos = edacurry.find_components_by_master(root, "nmos1")
# For each NMOS transistor found...
for component in nmos:
    # Replace its master model with a low-Vth version.
    component.master = "nmos_low_vth"
# Output the new netlist...

```

Listing 9. Replacing all NMOS transistors with low-Vth variants to simulate technology options.

As shown in Listing 9, transistor instances can be reassigned to alternative device models programmatically. This enables evaluation of low-threshold-voltage (low-Vth) options for speed improvement or energy reduction. This is an essential task when optimizing for modern CMOS nodes.

A further important application is the simulation of process-induced variability. Threshold voltage (vth) variation is a significant source of performance degradation in scaled technologies. Although some analog simulators support fundamental Monte Carlo analyses, they often impose rigid constraints on parameter distributions or require extensive manual setup. With EDACurry, designers can inject fully customized variability models by dynamically modifying transistor parameters according to arbitrary statistical distributions.

```

import numpy as np, edacurry, os
# Parse the ELDO file.
root = edacurry.parse_eldo("eldo/circ_base/opamp.cir")
# Settings for the Monte Carlo simulation parameters.
num_samples = 10 # Number of samples

```

```

vth_mean = 0.4 # Mean threshold voltage
vth_sigma = 0.02 # Standard deviation
# Generate random vth values for each sample.
vth_samples = np.random.normal(
    loc=vth_mean, scale=vth_sigma, size=num_samples)
# Get all NMOS devices.
devs = edacurry.find_components_by_master(root, "nmos1")
for idx, vth_value in enumerate(vth_samples):
    # Update VTH parameter for each NMOS.
    for nmos in devs:
        vth_param = edacurry.find_parameter(nmos, "VTH")
        vth_param.right = edacurry.Double(vth_value)
    # Output the Monte Carlo variant netlist...

```

Listing 10. Assigning randomized VTH values to NMOS transistors using a Gaussian distribution for Monte Carlo analysis.

In Listing 10, each netlist variant is generated by sampling a new VTH value for NMOS devices from a Gaussian distribution. Such randomization enables realistic yield analysis, parametric robustness evaluation, and stress testing. Unlike traditional built-in simulator sweeps, EDACurry provides fine-grained control, allowing for large-scale, programmable generation of statistically perturbed netlists.

By leveraging these capabilities, EDACurry empowers analog designers to perform advanced design space exploration tasks that are often tedious or infeasible with manual methods.

B. Injection of different classes of defect models

A major application of the EDACurry framework is the systematic injection of analog defect models into transistor-level circuits [28]. While the framework currently does not implement defect localization based on failure probabilities, future extensions could adopt state-of-the-art sampling techniques [29]. The IEEE P2427 draft standard provides guidelines for building and applying defect models to evaluate the fault coverage of analog circuits [30]. Defect modeling and injection remain active research areas, especially at the transistor level.

Traditionally, defects are injected into analog circuits using constructs like the .ALTER statement in Eldo, enabling individual fault activation during simulation. Fault injection campaigns require activating one fault at a time and running targeted simulations to compute accurate coverage metrics.

The defect injection flow begins by parsing a SPICE-based analog netlist using the appropriate front-end, mapping the circuit into the internal AST. From there, the framework can add, remove, or replace nodes, components, or entire subcircuits to introduce defect behavior. Another recent trend is implementing the defect templates at a behavioral level, with languages such as Verilog-A [28], which allows the modeling of complex fault models following the guidelines of the IEEE P2427 standard and can be easily integrated with existing design flows.

1) *Verilog-A implementation of generic defect models:* Using Verilog-A defect templates to represent different classes of defect models allows maintaining the portability of the defects across different analog simulators by changing only small parts of the original netlist [31]. These manipulations required to use the defect templates can be performed automatically by the EDACurry framework as described in the following. The associated testbench, written in SPICE, Eldo, or Spectre, is also modified to instantiate Verilog-A defect templates, enabling controlled fault activation during simulation.

A key component of the defect injection process is the Verilog-A fault wrapper library, which provides behavioral models for common transistor-level defects. Each wrapper dynamically selects the failure mode to simulate based on a failmode parameter associated with the device instance. Listing 11 shows a compact version of this Verilog-A wrapper, supporting fault-free behavior, drain opens, and gate-drain shorts.

```
module anabasis_pmos1 (D, G, S, B);
  inout D, G, S, B;
  electrical D, G, S, B;
  parameter real l = 0.35e-6, w = 10.0e-6, ...;
  parameter integer failmode = 0;
  parameter real ropen = 1.0e9, rshort = 0.1;
  generate
    case (failmode)
      (0): begin // FAULT-FREE
        pmos1 #(.l(l), .w(w), ...) core (D, G, S, B);
      end
      (1): begin // DRAIN OPEN
        electrical DD;
        pmos1 #(.l(l), .w(w), ...) core (DD, G, S, B);
        resistor #(.r(ropen)) defect (D, DD);
      end
      (5): begin // GATE-DRAIN SHORT
        pmos1 #(.l(l), .w(w), ...) core (D, G, S, B);
        resistor #(.r(rshort)) defect (G, D);
      end
      default: begin // DEFAULT FAULT-FREE
        pmos1 #(.l(l), .w(w), ...) core (D, G, S, B);
      end
    endcase
  endgenerate
endmodule
```

Listing 11. Compact Verilog-A wrapper for MOS transistor fault modeling. Each mode is selected via the failmode parameter.

Once the Verilog-A templates are available, the next step is to modify the device netlist to integrate these templates. The Device Under Test (DUT) netlist is modified to enable defect activation to include failure mode parameters, as shown below in Listing 12.

```
subckt OPAMP1_WRAPPED (inn inp out pd xpd vdda vssa)
  parameters failmode_mpd11=0 failmode_mpd12=0
  ...
  mpd11 (net56 inn net31 net31) anabasis_pmos1
  ... failmode = failmode_mpd11
  mpd12 (net13 inp net31 net31) anabasis_pmos1
  ... failmode = failmode_mpd12
ends OPAMP1_WRAPPED
```

Listing 12. OPAMP subcircuit modeled in SPECTRE that contains the necessary transformations in order to use the defect template wrappers.

The testbench is also automatically adapted to sweep through the injected defects for systematic coverage evaluation. The instrumented testbench is shown in Listing 13.

```
simulator lang=spectre
// Include the fault templates.
ahdl_include "fault_models.va"
// Global node for ground.
global 0
// Testbench parameters.
parameters VDD=3.3 VSS=0.0 NPD=10 FUP=10G FLO=1
parameters failmode_mpd11=0 failmode_mpd12=0
// Testbench
SSA (ssa 0) vsource dc=VSS type=dc
DDA (dda 0) vsource dc=VDD type=dc
XPD (xpd 0) vsource dc=VDD type=pwl \
  wave=[0 VDD 2u VDD 2.1u VSS]
PD (pd 0) vsource dc=VSS type=pwl \
  wave=[0 VSS 2u VSS 2.1u VDD]
INP (inp 0) vsource dc=(VDD/2) type=sine sinedc=(VDD/2) \
  ampl=500m freq=1M
Cload (out 0) capacitor c=50f
// Instantiate the Device Under Test.
DUT (out inp out pd xpd dda ssa) OPAMP1_WRAPPED \
```

```
failmode_mpd11=failmode_mpd11 \
failmode_mpd12=failmode_mpd12
// Define transient sweep for first failure mode.
sweep_dpi_power sweep param=failmode_mpd11 \
  values=[0 1 2 3 4 5 6 7 8 9 10] {
  tran tran stop=2.3u errpreset=conservative
  write="spectre.ic" writefinal="spectre.fc" \
  annotate=status maxiters=5
}
// Define transient sweep for second failure mode.
sweep_dpi_power sweep param=failmode_mpd12 \
  values=[0 1 2 3 4 5 6 7 8 9 10] {
  tran tran stop=2.3u errpreset=conservative
  write="spectre.ic" writefinal="spectre.fc" \
  annotate=status maxiters=5
}
```

Listing 13. Spectre testbench sweeping different activated defects for components mpd11 and mpd12.

Each sweep selectively activates one fault at a time, allowing efficient calculation of fault coverage metrics.

As described in [31], ten defect modes are supported through the Verilog-A templates: Fault-free (0), drain open (1), gate open (2), source open (3), bulk open (4), gate-drain short (5), gate-source short (6), gate-bulk short (7), bulk-drain short (8), bulk-source short (9), and drain-source short (10).

C. Exporting analog circuits as XML and JSON

Another major feature of EDACurry is the ability to export the internal representation into XML or JSON formats. This is achieved through two dedicated back-ends that serialize the unified AST. The XML output is highly structured for machine parsing and tool integration, while the JSON output is more human-readable and easily processed in dynamic environments like Python.

Both formats preserve the hierarchical tree structure of the internal AST, ensuring that no semantic information is lost during serialization. EDACurry's exported descriptions are fully language-agnostic, meaning they capture all netlist elements independent of whether the input was SPICE, Eldo, or Spectre.

An example XML description of the OPAMP subcircuit is shown in Listing 14. This format mirrors the internal data structures, making it suitable for programmatic manipulation, validation, or transformation pipelines.

Serialization into XML allows the modified netlist to be reused in external tools for further manipulation, validation, or documentation. Listing 14 shows a fragment of the XML generated through our framework for the OPAMP subcircuit.

```
<circuit name="opamp.cir">
  <content>
    <subckt name="OPAMP1">
      <nodes>
        <node name="inn"/>
        <node name="inp"/>
        <node name="out"/>
        <node name="pd"/>
        <node name="xpd"/>
        <node name="vdda"/>
        <node name="vssa"/>
      </nodes>
      <content>
        <component name="xd1" master="prim_nwd">
          <nodes>
            <node name="vssa"/>
            <node name="vdda"/>
          </nodes>
          <parameters>
            <parameter type="param_assign">
              <left>
```

```

<identifier name="AREA"/>
</left>
<right>
<number type="double" value="6.13907e-9"/>
</right>
</parameter>
...

```

Listing 14. XML fragment of the OPAMP subcircuit generated through the EDACurry framework.

JSON support is particularly valuable for lightweight workflows. Because JSON can be easily parsed into Python dictionaries, designers can search, modify, or analyze circuits directly using Python libraries without depending on EDACurry at every step. This enables workflows where an analog netlist is read, manipulated using EDACurry, exported to JSON, and processed further by standalone Python scripts.

This intermediate serialization capability is fundamental to building flexible, modular manipulation pipelines, especially in fault injection, test generation, or design-space exploration scenarios.

D. Graphically representing netlists as trees

Analog designers working at the transistor level often face difficulties understanding the structure of complex netlists. There is a strong demand for tools that can graphically represent netlists, making their hierarchical organization and interconnections easier to comprehend. Graphical visualization is handy for netlists containing multiple circuits, where manual inspection becomes impractical.

Rather than showing full transistor-level connectivity, the graphical tree highlights the logical organization and hierarchy of the netlist. This provides an effective tool for planning subsequent manipulation steps using the EDACurry framework.

This visualization is achieved by exporting the internal AST to JSON using the framework’s dedicated back-end, and then converting the JSON into a graphical representation using the Graphviz [32] library and the Graph Description Language (DOT) language.

```

.PARAM length=0.053u width=0.047u
c5 2 0 100p
r5 2 0 value=3.3k
xnw 1 2 nw l=length w=width
.SUBCKT nw n1 n2 l=length w=width
.PARAM rsh=1100
.SUBCKT rngps n3 n4 l2=w2 w2=w2
.PARAM rsh2=5.0
r2 n3 n4 'rsh2*(l2/w2)'
.ENDS rngps
xrngps n1 mid rngps l2=l*2 w2=w*2
r1 mid n2 'rsh*(l/w)'
.ENDS nw
.ALTER a1
r5 1 2 10k
c5 2 0 100p
.AC dec 15 200 1500meg
.END
.TRAN 2e-6 2e-3

```

Listing 15. Simple netlist modeled with the Eldo language.

For example, consider the Eldo netlist shown in Listing 15. This simple circuit illustrates the primary elements typically found in analog netlists, including subcircuits, parameters, components, and analysis statements.

By parsing this Eldo netlist with EDACurry and exporting it to JSON, a Python script can convert it into DOT format

for graphical visualization. The resulting structure is rendered using Graphviz.

Figure 6 shows a graphical representation of the netlist in Listing 15 produced by dot, based on the information retrieved by the EDACurry framework. In the graph, blue nodes represent parameters, yellow nodes represent components, and red nodes represent control statements. Lilac nodes represent the master of a given component. This hierarchical view significantly simplifies the comprehension and manipulation of complex analog netlists.

VI. CONCLUSIONS

Transistor-level netlist manipulation remains a critical yet underserved area in analog design automation, often constrained by proprietary tools and fragmented workflows. EDACurry addresses these limitations by introducing an open-source, language-independent framework that unifies parsing, analysis, and transformation of netlists across major SPICE-derived dialects, including Eldo and Spectre.

At the core of EDACurry lies a robust internal representation that abstracts away syntax-specific differences while preserving the full semantics of the original netlists. This unified model enables seamless manipulation routines, including structural edits, parameter sweeping, subcircuit wrapping, and defect injection. Through its C++ core and Python bindings, EDACurry provides high performance and accessibility, supporting a broad range of use cases from interactive prototyping to automated toolchains.

We demonstrated the framework’s versatility through practical applications such as design space exploration, analog fault modeling, tools pipelining through intermediate formats like XML and JSON, and graphical netlist introspection. These use cases highlight EDACurry’s capacity to automate previously manual and error-prone tasks, integrate with simulation flows, and serve as a foundation for domain-specific tools.

Looking ahead, we aim to expand support to additional netlist dialects, extend backend capabilities, and integrate machine learning models for intelligent design assistance and fault analysis. We also plan to develop higher-level APIs for rapid composition of analysis pipelines. By offering transparency, extensibility, and cross-language operability, EDACurry lays the groundwork for a new generation of analog design tools centered on reusability, portability, and semantic precision.

REFERENCES

- [1] A. Vladimirescu, *The Spice Book*. USA: John Wiley & Sons, Inc., 1994.
- [2] R. Pratap, V. Agarwal, and R. K. Singh, “Review of various available spice simulators,” in *2014 International Conference on Power, Control and Embedded Systems (ICPES)*, 2014, pp. 1–6.
- [3] K. S. Kundert and P. Gray, *The Designer’s Guide to Spice and Spectre*. USA: Kluwer Academic Publishers, 1995.
- [4] N. Dall’Ora, S. Azam, E. Fraccaroli, A. Alberts, and F. Fummi, “A common manipulation framework for transistor-level languages,” in *2021 Forum on specification & Design Languages (FDL)*, 2021, pp. 01–07.
- [5] J. Wenzel, “Pybind11 — seamless operability between c++11 and python,” <https://readthedocs.org/projects/pybind11/downloads/pdf/stable/>, 2024, accessed: 2025-04-23.

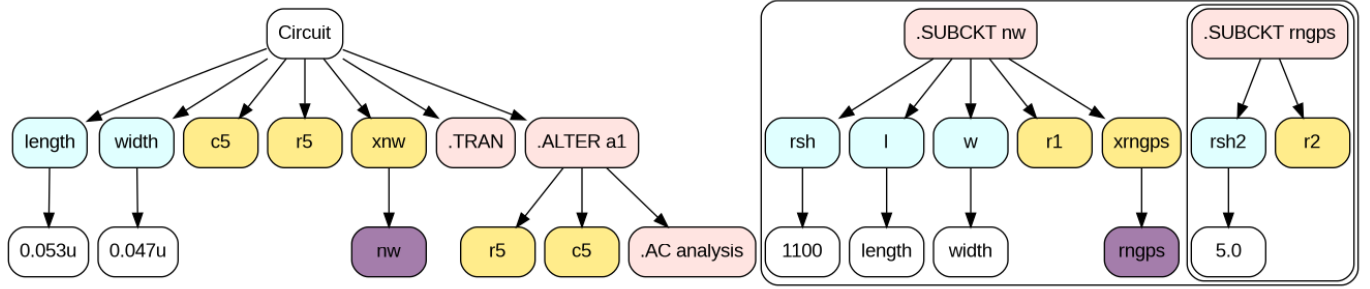


Figure 6. Graphical visualization of the netlist presented in Listing 15 produced through the EDACurry framework.

- [6] R. Pratap, V. Agarwal, and R. K. Singh, "Review of various available spice simulators," in *2014 International Conference on Power, Control and Embedded Systems (ICPACES)*, 2014, pp. 1–6.
- [7] P. W. Tuinenga, *Spice: A Guide to Circuit Simulation and Analysis Using PSpice*, 2nd ed. USA: Prentice Hall PTR, 1991.
- [8] Accellera, "Verilog-AMS Language Reference Manual," <https://www.accellera.org/images/downloads/standards/v-ams/VAMS-LRM-2-4.pdf>, 2014.
- [9] T. Parr, *The Definitive ANTLR 4 Reference*, 2nd ed. Pragmatic Bookshelf, 2013.
- [10] Siemens, "Spect2el script — Eldo User's Manual Release AMS 2009.2," https://www.eet.bme.hu/~szalai/Download/eldo_ur.pdf, 2009, accessed: 2025-04-23.
- [11] K. Technologies, "Netlist Translator for SPICE and Spectre," <http://edadownload.software.keysight.com/eedl/ads/2011/pdf/netlist.pdf>, 2002, accessed: 2025-04-23.
- [12] K. Bard, B. Dewey, M.-T. Hsu, T. Mitchell, K. Moody, V. Rao, R. Rose, J. Soreff, and S. Washburn, "Transistor-level tools for high-end processor custom circuit design at ibm," *Proceedings of the IEEE*, vol. 95, no. 3, pp. 530–554, 2007.
- [13] D. Batas and H. Fiedler, "A python interface for spice-based simulations," in *ICSES 2010 International Conference on Signals and Electronic Circuits*, 2010, pp. 161–164.
- [14] D. Batas, S. Knaak, and H. Fiedler, "Spicedim - a cad tool for supporting circuit sizing of analog cmos circuits," in *2010 International Conference on Microelectronics*, 2010, pp. 351–354.
- [15] Y. Li, Y. Zhang, J. Liu, J. Gong, J. Wang, J. Lai, X. Tao, and G. Qu, "AutoTEA: An automated transistor-level efficient and accurate design tool for FPGA design," *Integration*, vol. 87, pp. 231–240, Nov. 2022.
- [16] C. University, "Sml: Spice manipulation language," <http://www.cs.columbia.edu/~sedwards/classes/2004/w4115-fall/reports/SML.pdf>, 2004, accessed: 2025-04-23.
- [17] T. Edwards, "Netgen 1.5," <http://opencircuitdesign.com/netgen/index.html>, 2017, accessed: 2025-04-23.
- [18] F. Salvaire, "Pyspice," <https://pyspice.fabrice-salvaire.fr/releases/v1.4/index.html>, 2017, accessed: 2025-04-23.
- [19] D. Vungarala, S. Alam, A. Ghosh, and S. Angizi, "Spicepilot: Navigating spice code generation and simulation with ai guidance," 2024. [Online]. Available: <https://arxiv.org/abs/2410.20553>
- [20] D. Fritchman, "An integrated circuit design framework for human, computer, and ml designers," Ph.D. dissertation, EECS Department, University of California, Berkeley, Dec 2023. [Online]. Available: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2023/EECS-2023-275.html>
- [21] —, "Vlsir - a modular framework for programming analog & custom circuits & layouts," in *Proceedings of the 2023 International Symposium on Physical Design*. New York, NY, USA: Association for Computing Machinery, 2023, p. 82–83.
- [22] S. Vinagrero Gutiérrez, G. Di Natale, and E.-I. Vatajelu, "Python framework for modular and parametric spice netlists generation," *Electronics*, vol. 12, no. 18, 2023.
- [23] Siemens, "Eldo User's Manual Release AMS 2009.2," https://www.eet.bme.hu/~szalai/Download/eldo_ur.pdf, 2009, accessed: 2025-04-23.
- [24] Cadence, "Spectre Circuit Simulator Reference Manual," https://ee.kpi.ua/~yv/edu/ok/book/spectre_refManual.pdf, 2020, accessed: 2025-04-23.
- [25] N. Bombieri, G. Di Guglielmo, F. Michele, F. Fummi, G. Pravaddelli, F. Stefanni, and V. Alessandro, "Hifsuite: Tools for hdl code conversion and manipulation," *EURASIP Journal on Embedded Systems*, vol. 2010, 01 2010.
- [26] S. Sunter and P. Sarson, "A/MS benchmark circuits for comparing fault simulation, DFT, and test generation methods," in *2017 IEEE International Test Conference (ITC)*. IEEE, Oct. 2017.
- [27] M. Schweikardt and J. Scheible, "Expert design plan: A toolbox for procedural automation of analog integrated circuit design," in *2022 18th International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD)*, 2022, pp. 1–4.
- [28] S. Azam, N. Dall'Ora, E. Fraccaroli, R. Gillon, and F. Fummi, "Analog defect injection and fault simulation techniques: A systematic literature review," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 1, pp. 16–29, 2024.
- [29] S. Sunter, K. Jurga, P. Dingenen, and R. Vanhooren, "Practical random sampling of potential defects for analog fault simulation," in *2014 International Test Conference*, 2014, pp. 1–10.
- [30] "IEEE Draft Standard for Analog Defect Modeling and Coverage," *IEEE P2427/D0.40, October 2024*, pp. 1–106, 2024.
- [31] N. Dall'Ora, S. Azam, E. Fraccaroli, R. Gillon, and F. Fummi, "Verilog-a implementation of generic defect templates for analog fault injection," in *Proceedings of the Great Lakes Symposium on VLSI 2023*, 2023, pp. 477–481.
- [32] J. Ellson, E. R. Gansner, E. Koutsofios, S. C. North, and G. Woodhull, *Graphviz and Dynagraph — Static and Dynamic Graph Drawing Tools*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 127–148.



Nicola Dall'Ora (Member, IEEE) is a postdoctoral researcher at the Department of Engineering for Innovation Medicine of the University of Verona (Italy). He received his Ph.D. degree in Computer Science from the University of Verona, Italy, in May 2023. His primary research focuses on defect/fault injection and simulation for different modeling levels, from transistor-level to multi-domain systems described at the behavioral level, to extract fault modes useful to guarantee the functional safety of embedded platforms.



Enrico Fraccaroli (Member, IEEE) is a postdoctoral researcher at the Department of Computer Science of the University of North Carolina at Chapel Hill (USA) since January 2022. He received his Ph.D. degree in computer science from the University of Verona, Italy, in May 2019. His research interests are the development of new methodologies for the efficient simulation and functional safety evaluation of embedded platforms composed of analog, digital, and network components.